

DATA266 Lab 1 — Team 6 Report

Team: PairProgramming_Team_06 — Rajesh Paruchuri, Siva Surya Chandran · **Course:** DATA 266, Fall 2026

GitHub repository: https://github.com/Paruchuri-Rajesh/DATA_266-GPULab1

Kaggle: data-266-fall-2026-gan-image-style-transfer — team public rank 11 (checked 2026-10-04)

This single report covers all three parts. Each member independently designed, coded, trained and evaluated their own models for every part; the comparison tables and joint analyses were written together. Every number below is taken from the committed `metrics_report.csv` / `full_metrics_report.csv` files and the executed notebooks in each member's folder.

Results at a glance

	Rajesh Paruchuri	Siva Surya Chandran
Part 1 — validation cross-entropy (lower is better)	0.711 (perplexity 2.04)	0.639 (perplexity 1.89)
Part 2 — best test accuracy	0.9523 (BiGRU + attention)	0.9581 (BiLSTM + attention)
Part 3 — course-script FID / MiFID	149.48 / 0.414	101.91 / 0.408
Part 3 — Kaggle public score	-74.9486	-45.9695 (team best; v3 entry, see Part 3)
Seed · hardware	266 · Apple M4 MacBook Air (MPS), all parts	1337 · Part 1: NVIDIA T4 (Colab, fp16); Part 2: Apple M5 (MPS); Part 3: NVIDIA RTX 4090 (bf16)

Submission layout

Folder	Executed notebook(s)	Saved weights	Other outputs
Part 1 — task1_llm/	rajesh_paruchuri/src/part1_llm.ipynb sivasurya_chandran/src/task1_gpt.ipynb	rajesh_paruchuri/checkpoints/best.pt sivasurya_chandran/checkpoints/gpt_best.pt, gpt_final.pt	loss curves, generated samples, metrics_report.csv, failure_analysis.md, results.md
Part 2 — task2_sentiment/	rajesh_paruchuri/src/part2_sentiment.ipynb sivasurya_chandran/src/task2_sentiment.ipynb	Rajesh: baseline_fasttext_full.pt, exp_a_bigru_attn_full.pt, exp_b_transformer_full.pt Siva: baseline.pt, experimental_1.pt, experimental_2.pt	confusion / ROC / PR / calibration plots, per-slice metrics, 20-error reviews
Part 3 — task3_gan/	rajesh_paruchuri/src/part3_cyclegan.ipynb, Part3_Evaluation_Script_run.ipynb sivasurya_chandran/src/task3_cyclegan.ipynb, Part3_Evaluation_Script_run.ipynb	Rajesh: checkpoints/cyclegan_full.pt Siva: checkpoints/G_AB.pt, G_BA.pt, D_A.pt, D_B.pt (final model); checkpoints/history/v3/ (Kaggle entry)	submission.csv, translated images, sample grids, loss curves, human-audit kit

Raw training logs are in `reproducibility/raw_logs/<member>/` and environment manifests in `reproducibility/manifests/`.

Part 1 — GPT-style character-level language model from scratch (TinyStories)

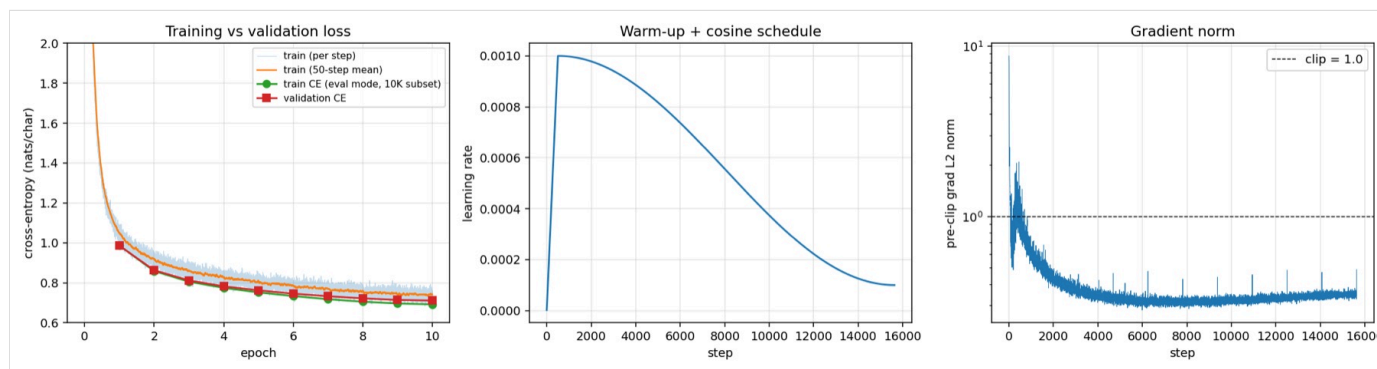
Approach

Both members implemented a decoder-only, GPT-style Transformer at the character level, writing causal multi-head self-attention, LayerNorm and the GELU feed-forward block by hand rather than using `nn.TransformerDecoder`. Stories were sampled from TinyStories (100,000 train / 10,000 validation), encoded with a character vocabulary and cut into fixed-length context windows. Training used AdamW with a linear warm-up followed by cosine decay; the model was evaluated on validation cross-entropy, perplexity, bits per character and next-character accuracy, and samples were generated with greedy and temperature sampling to study fluency and failure modes.

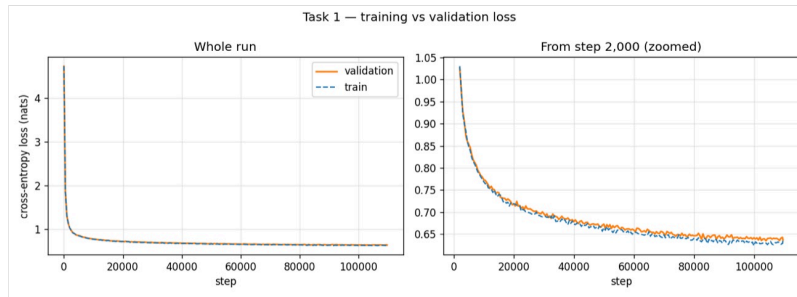
- **Rajesh** used a *story-disjoint* split (no story appears in both train and validation) and a longer 192-character context, with a weight-tied output head.
- **Siva** trained a wider model (8 heads, width 256) with fp16 mixed precision on a Colab T4 and an untied output head.

Architecture, parameters and results

Metric	Rajesh Paruchuri	Siva Surya Chandran
Architecture	Decoder-only GPT; hand-written LayerNorm / causal MHA / GELU FFN; pre-norm; weight-tied head	Decoder-only GPT; hand-written causal MHA / LayerNorm / GELU FFN; pre-norm; separate LM head
Layers / heads / width / context	6 / 6 / 192 / 192 chars	6 / 8 / 256 / 128 chars
Data split	Story-disjoint 100,000 / 10,000, non-overlapping windows, seed 266	100,000 / 10,000 stories (seed 1337), non-overlapping 128-char windows
Optimiser & schedule	AdamW (0.9, 0.95), wd 0.1, lr 1e-3, 500-step warm-up + cosine to 1e-4	AdamW (0.9, 0.95), wd 0.01, lr 3e-4, 500-step warm-up + cosine to 3e-5
Batch / dropout / epochs	64 / 0.1 / 10	64 / 0.1 / 10, fp16
Parameters	2,719,488	4,827,648
Train CE (eval mode) / Val CE	0.6920 / 0.7111	0.6303 / 0.6385
Perplexity / bits per char	2.036 / 1.026	1.894 / 0.921
Generalisation gap (val – train)	+0.019	+0.008
Top-1 next-char accuracy	0.7746	0.7955
Distinct-1/2/3 (sampled)	0.387 / 0.801 / 0.935 (T = 0.7)	0.252 / 0.676 / 0.886 (T = 0.8)
Repeated 4-gram rate (greedy / sampled)	0.396 / 0.005	0.267 / 0.006
Grad-norm mean / max · NaNs · spikes	0.386 / 8.75 · 0 · 0	0.343 / 7.77 · 0 · 0 (41 fp16 overflow steps skipped by the scaler)
Throughput (train / generation tokens/s)	18,282 / 100	133,346 / 217
Peak memory · train time	3,308 MB (MPS) · 11,173 s	944 MB (CUDA) · 6,736 s



Rajesh — training / validation loss curves (outputs/loss_curves_full.png).



Siva — training / validation loss curves (outputs/loss_curves.png).

Analysis

- **Fit.** Both models learn spelling and local grammar well (77–80% next-character accuracy) and train stably with warm-up + cosine: no NaNs, no loss spikes, small generalisation gaps. Neither model overfits, so both could use more capacity.
- **Comparison.** Siva's larger model (4.8M vs 2.7M parameters) reaches lower validation CE (0.639 vs 0.711). This is not strictly like-for-like: Rajesh's story-disjoint split is harder, and his longer context should help coherence. fp16 on the T4 trained about 7× faster per second than fp32 on the M4; the GradScaler skipped only 41 of 109,640 steps.
- **Shared failure modes.** Greedy decoding falls into repetition loops (e.g. *"The sun was shining and the sky was blue. The sun was shining..."*); characters change names or pronouns once they leave the context window (*"a little boy named Timmy"* → three sentences later *"Lily looked at the tree..."*); high temperature produces invented words (*"a bloll on the floor"*) and impossible events.
- **Next steps.** Longer context or BPE tokens, top-k / nucleus sampling or a repetition penalty, a KV cache for generation, and a larger model.

Part 2 — Yelp polarity sentiment classification

Approach

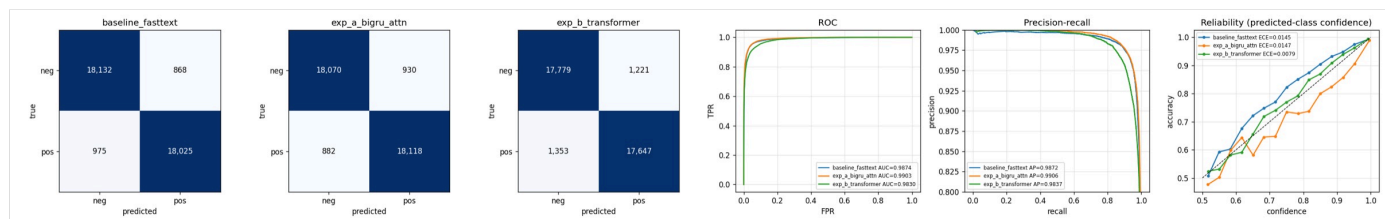
Each member trained one baseline and two experimental models on Yelp Review Polarity with word embeddings learned from scratch (no pre-trained vectors). Reviews were cleaned and tokenised, a vocabulary built on the training split, and a validation split held out from training for model selection; the official 38,000-review test set was used only for final evaluation. Beyond accuracy, both members report macro/micro/weighted precision-recall-F1, ROC-AUC, PR-AUC, MCC, Brier score, expected calibration error, bootstrap 95% confidence intervals, McNemar's test against their own baseline, per-slice metrics (length, negation, contrast, non-English), and a manual review of 20 misclassified reviews.

- **Rajesh** (539,941 train / 20,000 val): fastText-style unigram + hashed-bigram bag (baseline); BiGRU + additive attention (Exp A); 2-layer Transformer encoder written from scratch (Exp B).
- **Siva** (503,979 train / 55,997 val): mean-pooled embeddings → MLP (baseline); multi-width CNN, kernels 3/4/5 (Exp 1); BiLSTM + additive attention (Exp 2).

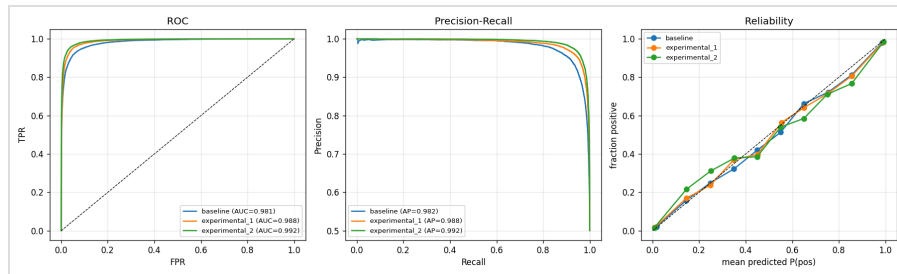
Architecture, parameters and results (test set, n = 38,000)

Metric	Rajesh: fastText bigram (baseline)	Rajesh: BiGRU + attn	Rajesh: Transformer	Siva: bag-of-embeddings (baseline)	Siva: CNN	Siva: BiLSTM + attn
Architecture	uni + hashed-bigram embedding bag → linear	BiGRU 128/dir + additive attention	2-layer encoder, 4 heads, [CLS]	mean-pooled embeddings → MLP (hidden 128)	1-D CNN, kernels 3/4/5 × 128, max-pool	BiLSTM 128/dir (packed) + additive attention
Key hyperparameters	emb 64, lr 2e-3, 4 ep	emb 128, lr 1e-3, 3 ep	emb 128, lr 5e-4, 3 ep	emb 128, lr 2e-3, dropout 0.3, ≤5 ep	emb 128, lr 1e-3, dropout 0.5, ≤5 ep	emb 128, lr 1e-3, dropout 0.3, ≤5 ep
Accuracy	0.9515	0.9523	0.9323	0.9332	0.9502	0.9581
Macro P / R / F1	0.9515 (all)	0.9523 (all)	0.9323 (all)	0.9332 (all)	0.9502 (all)	0.9581 (all)
ROC-AUC / PR-AUC	0.9874 / 0.9872	0.9903 / 0.9906	0.9830 / 0.9837	0.9812 / 0.9816	0.9883 / 0.9882	0.9921 / 0.9923
MCC	0.9030	0.9046	0.8645	0.8665	0.9004	0.9161
Brier / ECE	0.0382 / 0.0145	0.0363 / 0.0147	0.0494 / 0.0079	0.0503 / 0.0082	0.0384 / 0.0105	0.0323 / 0.0124
95% CI accuracy	[0.9496, 0.9535]	[0.9502, 0.9545]	[0.9298, 0.9346]	[0.9309, 0.9359]	[0.9479, 0.9523]	[0.9560, 0.9601]
McNemar vs own baseline	—	exact p = 0.44	exact p = 4×10^{-74}	—	p < 10^{-16}	p < 10^{-16}
Worst slice macro-F1	0.875 (non-English, n = 133)	0.810 (non-English)	0.781 (non-English)	0.924 (contrast)	0.945 (contrast)	0.953 (contrast)
Params · train time	20.26M · 1,195 s	7.20M · 17,403 s	7.26M · 9,084 s	6.42M · 173 s	6.60M · 791 s	6.70M · ≈10,600 s
Hardware	Apple M4, MPS			Apple M5, MPS		

Macro, micro and weighted P/R/F1 coincide because the test set is exactly balanced. "Contrast" = reviews containing but / however / although. The worst slice differs between members because the slices were defined differently.



Rajesh — confusion matrices, ROC, PR and calibration curves for all three models (outputs/confusion_roc_pr_calibration_full.png).

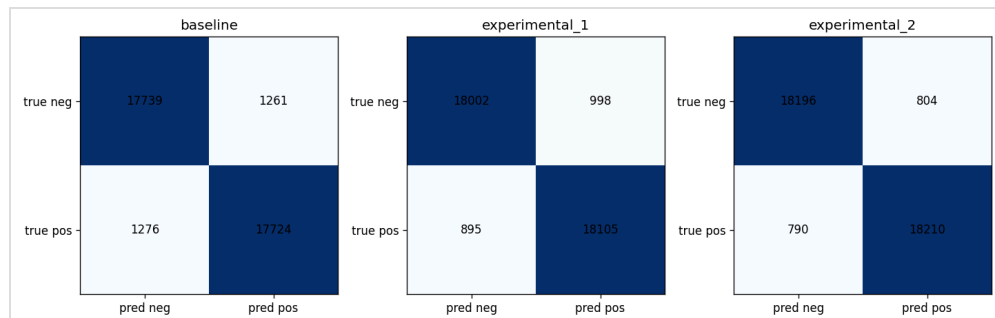


Siva — test ROC / PR / calibration curves (outputs/test_curves.png).

Throughput, memory and confusion matrices (test set, n = 38,000)

Metric	Rajesh: fastText bigram (baseline)	Rajesh: BiGRU + attn	Rajesh: Transformer	Siva: bag-of-embeddings (baseline)	Siva: CNN	Siva: BiLSTM + attn
Training examples/sec	1,807	93	178	14,535	3,188	132
Test inference examples/sec	56,946	7,178	4,467	105,134	8,354	2,959
Peak memory (MPS driver, MB)	2,787	4,032	10,130	2,810	2,828	4,006
Confusion matrix TN / FP / FN / TP	18,132 / 868 / 975 / 18,025	18,070 / 930 / 882 / 18,118	17,779 / 1,221 / 1,353 / 17,647	17,739 / 1,261 / 1,276 / 17,724	18,002 / 998 / 895 / 18,105	18,196 / 804 / 790 / 18,210

Positive = positive review. Peak memory is the MPS driver allocation reported by `torch.mps.driver_allocated_memory()` for both members; Rajesh's CSV also records the tensor-only peak and the process RSS. Siva's BiLSTM throughput is from the logged training time, which includes about 8,500 s of laptop sleep. Values are from each member's `metrics_report.csv`.



Siva — test confusion matrices for the three models (outputs/test_confusion.png).

Analysis

- **Best model.** Siva's BiLSTM + attention is the team's best (accuracy 0.9581); its CI [0.9560, 0.9601] does not overlap Rajesh's best (BiGRU, [0.9502, 0.9545]). Recurrent encoders with attention pooling came out on top in both lineups.
- **Word order helps, and bigrams capture much of it.** Siva's bag → CNN → BiLSTM went 0.933 → 0.950 → 0.958, with the biggest gains on negation and contrast slices (contrast error 7.5% → 5.4% → 4.65%). Rajesh's bigram baseline (0.9515) is almost two points above the unigram baseline and statistically indistinguishable from his BiGRU (McNemar $p = 0.44$) at a fraction of the cost.
- **Transformer from scratch.** The weakest experimental model (0.9323): it overfits after epoch 1 without pre-training, though it is the best-calibrated model (ECE 0.008).
- **Calibration vs accuracy.** The most accurate models are slightly over-confident; temperature scaling on validation would fix this cheaply.
- **Error reviews (20 errors each).** Rajesh: 6 errors came from his own preprocessing (deleted prices, lost case/emphasis, the stopword "just", split accents) and 3 were label noise. Siva: 8 contrast clauses ("Food is good but the portions are small..."), 4 label noise, 3 truncation at 256 tokens, 2 past-vs-present, 2 with no sentiment signal, 1 negated negative.
- **Next steps.** Temperature scaling, head + tail truncation, a Unicode-aware tokeniser, contrast/negation scope marking, and an ensemble (averaging Rajesh's baseline and BiGRU gave 0.9572 post hoc).

Part 3 — CycleGAN photo ↔ Monet style transfer

Approach

Both members trained an unpaired CycleGAN (two ResNet generators $G_{\text{photo} \rightarrow \text{Monet}}$, $G_{\text{Monet} \rightarrow \text{photo}}$ and two 70×70 PatchGAN discriminators) on the Kaggle competition data (300 Monet paintings, ~7,000 photos). The objective combines an LSGAN adversarial loss, an L1 cycle-consistency loss ($\lambda_{\text{cyc}} = 10$) and an identity loss. Because 300 paintings is a very small target set, both used DiffAugment to stop the discriminator memorising the paintings, an exponential moving average (EMA) of generator weights for evaluation, and resize-convolution upsampling to avoid checkerboard artifacts. Final models were scored with the course-provided `Part3_Evaluation_Script.ipynb` (FID and MiFID), and translations were also evaluated with KID, precision/recall, LPIPS, content similarity and cycle L1.

Architecture, parameters and results

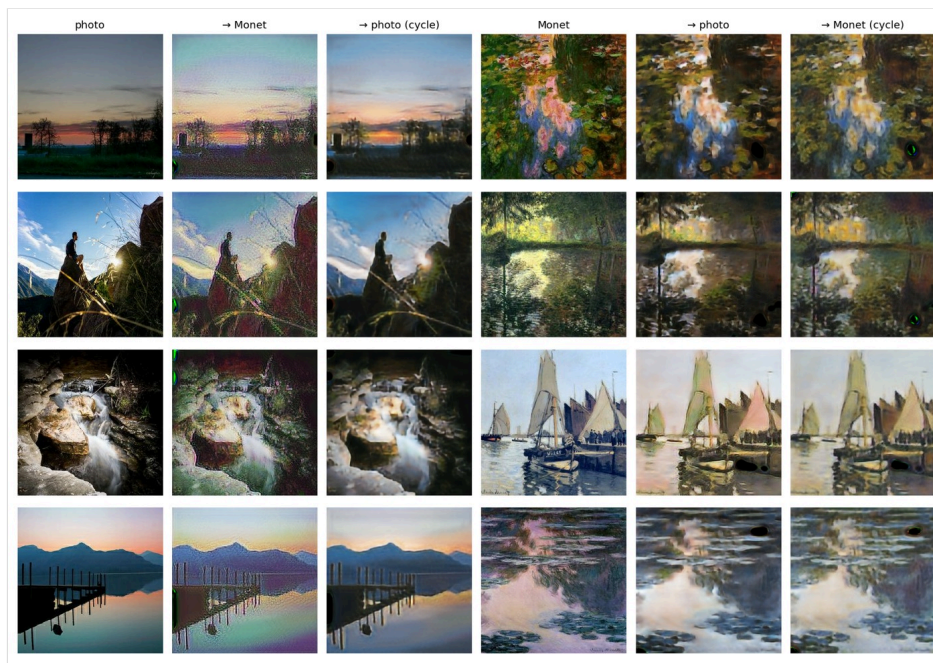
Metric	Rajesh Paruchuri	Siva Surya Chandran
Generators / discriminators	ResNet, 6 blocks, ngf 48, bilinear + conv decoder / spectral-norm 70×70 PatchGAN	ResNet, 9 blocks, ngf 64, nearest-resize + conv decoder / InstanceNorm 70×70 PatchGAN
Training	DiffAugment (colour / translation / cutout), EMA, 128-px crops, 80 epochs × 300 iterations, LSGAN, λ_{cyc} 10, λ_{id} 5	DiffAugment (translation), EMA, 256-px crops, warm start from v2 (50K steps) + 270 epochs × 1,000 iterations, LSGAN, λ_{cyc} 10, λ_{id} 2.5; checkpoint chosen on held-out photos
Parameters	14,355,336	28,285,832
Course-script FID / MiFID	149.48 / 0.414	101.91 / 0.408
Course-script FID per direction (photo→Monet / Monet→photo)	129.21 / 169.76	96.96 / 106.86
Kaggle public score (submission.csv)	−74.9486	−45.9695 (team best) — v3 entry, 1,91.537,0.402; the final model above scored −51.1585
Own-eval FID photo→Monet / Monet→photo	105.72 / 148.80 (all images)	83.00 / 89.68 (1,000 held-out photos vs 300 paintings)
KID photo→Monet / Monet→photo	0.0308 / 0.0776	0.0126 / 0.0257
Precision / recall (photo→Monet)	0.247 / 0.523	0.576 / 0.630
Cycle L1 (photo / Monet)	0.0774 / 0.0775	0.1053 / 0.0841
LPIPS input vs translation	0.523	0.420
Content cosine (photo→Monet)	0.373 (VGG16 relu4_3)	0.732 (Inception-v3 pool)
Final losses G-adv / D_A / D_B / cycle / identity	0.645 / 0.214 / 0.216 / 0.358 / 0.373	1.584 / 0.063 / 0.035 / 0.159 / 0.149
Grad norm G / D (mean) · NaNs	44.2 / 5.5 · 0	15.3 / 9.3 · 0
Human audit (blinded, 30 samples, 2 raters)	Not completed — audit kit prepared in <code>outputs/human_audit/</code> ; ratings were not collected	3.78 / 5 (style 3.65, content 3.93, artifacts 3.75); quadratic κ 0.24, 83% within-1 agreement
Train time · throughput · peak memory	9,417 s · 5.1 img/s · 1,009 MB (MPS pool)	24,770 s (final run) · 10.9 steps/s · 19,863 MB (CUDA)
Hardware	Apple M4, MPS	NVIDIA RTX 4090 (lab PC), CUDA, bf16

Kaggle score = $-(\text{FID} + \text{MiFID})/2$ as observed on the leaderboard. Own-eval FIDs use each member's own image sets and feature extractors and are not comparable across members; the course-script rows are directly comparable.

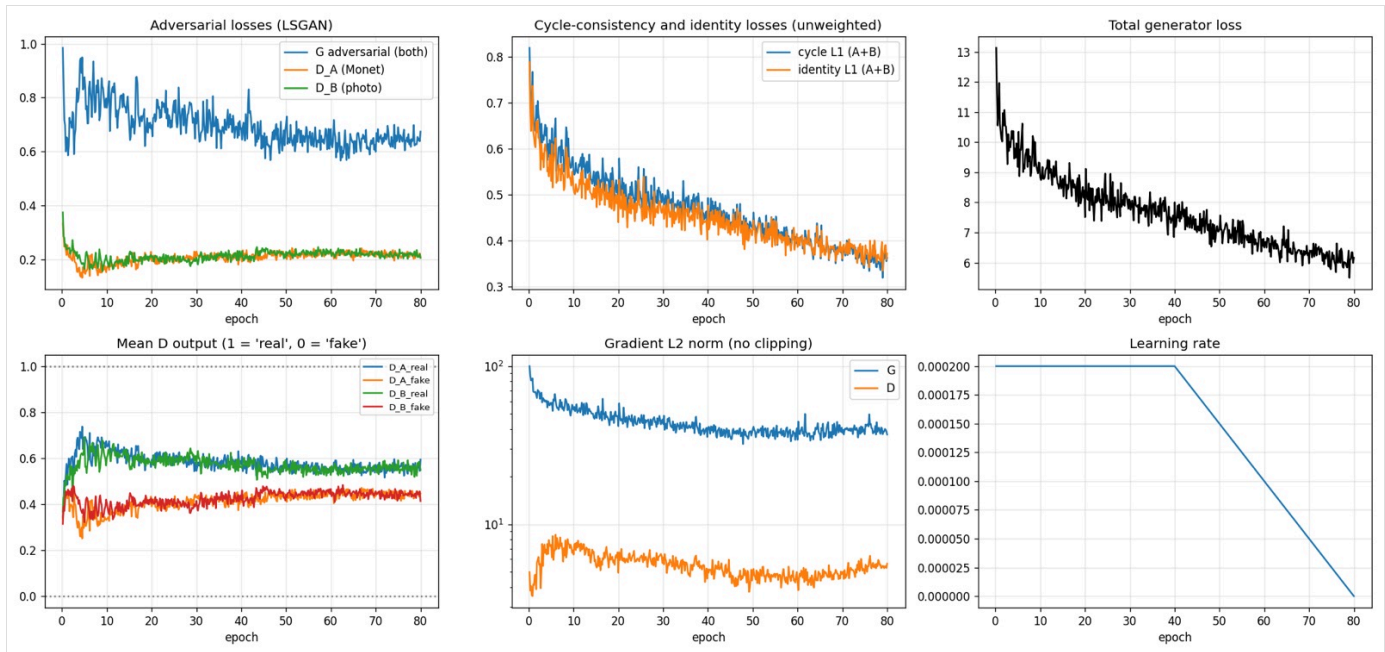
Siva's Kaggle entry. Agreed with the TA as the entry to report. `submission.csv` comes from Siva's v3 model (`checkpoints/history/v3/G_AB.pt`, best epoch 11; v3 warm-started from v2 and trained 12K more steps on an Apple M5), applied to all 7,038 competition photos (`outputs/history/v3/pred_B2A`). It was scored before the course script was shared, with `evaluate_competition_stats.py`: photo→Monet FID of all 7,038 translations against the competition's `real_stats.npz`, which gives FID 91.537 / MiFID 0.402 and reproduces from the committed images. Because it uses one direction and many more images, its FID is not comparable with the course-script rows above. Details: `task3_gan/sivasurya_chandran/results.md`.



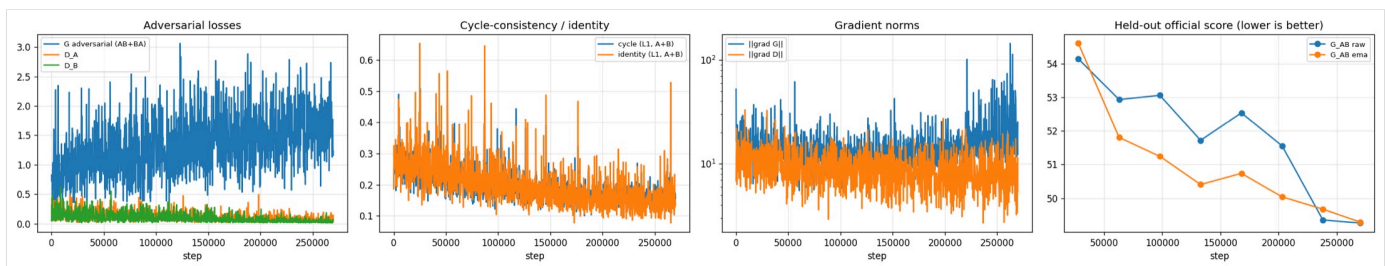
Siva — photo → Monet on held-out photos. Rows: input photo, translation, cycle reconstruction. Note the watermark ("© 2016 Thomas J. Astle") surviving translation.



Rajesh — both directions with cycle reconstructions. Small dark/green "hidden code" blobs are visible in several outputs.



Rajesh — CycleGAN loss curves.



Siva — CycleGAN loss curves (final run).

Analysis

- **Same scorer, direct comparison.** Course-script FID 101.91 (Siva) vs 149.48 (Rajesh). Siva's model has about twice the parameters (28.3M vs 14.4M) and saw far more training at full 256-px resolution (50K + 270K steps vs 24K steps on 128-px crops), which explains most of the gap. Rajesh's spectral-norm discriminators gave a steadier min-max balance with half the parameters, but the model was under-trained (cycle loss still falling at epoch 80).
- **Monet → photo is the weak direction for both** (FID 106.9 vs 97.0 for Siva; 169.8 vs 129.2 for Rajesh). The 300 paintings include still lifes and interiors with no photographic counterpart, and inventing photographic detail from a painting is under-determined.
- **Stability.** Neither run produced NaNs (0 in 270,000 steps for Siva). For Siva, EMA weights won 7 of 9 held-out checks; once the learning rate decayed near zero the raw weights won. Two extra experiments (a UVCGAN-style model, which collapsed to identity, and a PatchNCE continuation) scored worse and were not used.
- **Failure modes.** Steganographic "hidden code" blobs that partly satisfy the cycle loss; bright light sources turned into white blobs with halos; out-of-domain content (watermarks, text, frames) copied through; grey photos given invented colour; dramatic skies washed out; Monet→photo outputs too dark or still painterly; border streaks from 128-px crop training.
- **Next steps.** Longer full-resolution training, a stronger or reweighted Monet→photo direction, noise in the cycle path against hidden codes, and masking watermarks in the training photos.

Reproducibility

- Clone github.com/Paruchuri-Rajesh/DATA_266-GPULab1; the README gives per-member setup and the exact command for every part.
- Rajesh's runs are config-driven (`src/config.json`); `LAB_SMOKE=1 jupyter nbconvert --to notebook --execute <notebook>` runs a smoke test that writes to separate `*_smoke` files. All three notebooks were re-run under smoke mode on 2026-10-03 without errors.
- Siva's runs are driven by `config.yaml` in each member folder; commands are in his `results.md` and `reproducibility/manifests/sivasurya_chandran.md`.
- Datasets (TinyStories, Yelp polarity, Kaggle Monet/photo) are downloaded by the notebooks / `tools/download_raw_data.py` / `task3_gan/download_data.py` and are not included in the submission.

References

- Vaswani, A., et al. (2017). Attention is all you need. NeurIPS.
- Eldan, R., and Li, Y. (2023). TinyStories: How small can language models be and still speak coherent English? arXiv:2305.07759.
- Zhang, X., Zhao, J., and LeCun, Y. (2015). Character-level convolutional networks for text classification. NeurIPS.
- Joulin, A., Grave, E., Bojanowski, P., and Mikolov, T. (2017). Bag of tricks for efficient text classification. EACL.
- Kim, Y. (2014). Convolutional neural networks for sentence classification. EMNLP.
- Bahdanau, D., Cho, K., and Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. ICLR.
- Zhu, J.-Y., Park, T., Isola, P., and Efros, A. A. (2017). Unpaired image-to-image translation using cycle-consistent adversarial networks. ICCV.
- Zhao, S., Liu, Z., Lin, J., Zhu, J.-Y., and Han, S. (2020). Differentiable augmentation for data-efficient GAN training. NeurIPS.
- Miyato, T., Kataoka, T., Koyama, M., and Yoshida, Y. (2018). Spectral normalization for generative adversarial networks. ICLR.
- Heusel, M., et al. (2017). GANs trained by a two time-scale update rule converge to a local Nash equilibrium. NeurIPS.
- Chu, C., Zhmoginov, A., and Sandler, M. (2017). CycleGAN, a master of steganography. arXiv:1712.02950.